ภาษา Javascript สมัยใหม่

What is JavaScript?

JavaScript was initially created to *“make webpages alive”*.

What is JavaScript?

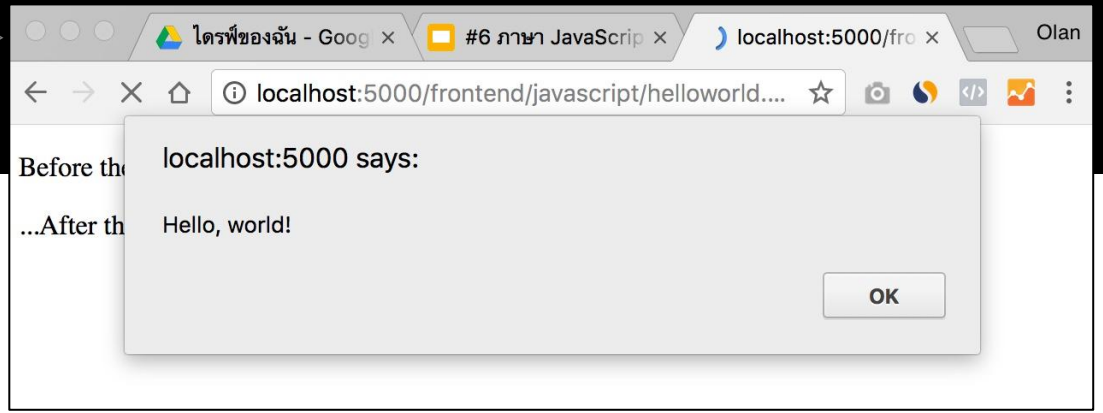
- Add new HTML to the page, change the existing content, modify styles.
- React to user actions, run on mouse clicks, pointer movements, key presses.
- Send requests over the network to remote servers, download and upload files (so-called AJAX and COMET technologies).
- Get and set cookies, ask questions to the visitor, show messages.
- Remember the data on the client-side (“local storage”).

The “script” tag

```
<!DOCTYPE HTML>
<html>
<body>
   <p>Before the script...</p>

   <script>
       alert('Hello, world!');
   </script>

   <p>...After the script.</p>
</body>
</html>
```



External scripts

```
<script src="main.js"></script>
```

CDN scripts

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/3.2.0/lodash.js"></script>
```

Several scripts

```
<script src="main.js"></script>
<script src="js/script.js"></script>
```

Split into two scripts

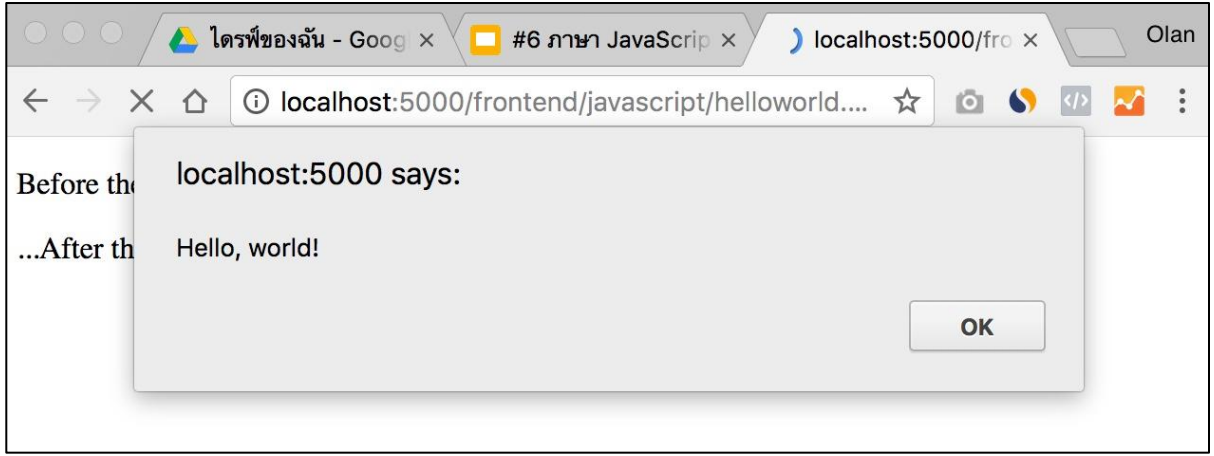
```
<script src="js/script.js"></script>
<script>
    alert(1);
</script>
```

The content is ignored, because src is set

```
<script src="main.js">
    alert(1); // the content is ignored, because src is set
</script>
```

alert

```
<script>
    alert('Hello, world!');
</script>
```



Developer Tools

share.olanlab.com/th

Share

เรียนรู้ แบ่งปันกับ Share

เคล็ดลับและเทคนิค

แนะนำ 9 เว็บไซต์แจกรูปฟรีแบบไม่ติดลิขสิทธิ์

แนะนำเว็บไซต์แจกรูปฟรี แบบไม่ติดลิขสิทธิ์สำหรับนักพัฒนาเว็บ (Web Developer) นักทำเนื้อหา (Content Creator) กราฟฟิค (Graphic)

Sep 10, 2017

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Audits

```
<!DOCTYPE html>
<html lang="th" xmlns="http://www.w3.org/1999/xhtml" xmlns:fb="http://ogp.me/ns/fb#">
  ▶<head>…</head>
…▼<body> == $0
    ▶<div class="navbar navbar-inverse navbar-fixed-top" role="navigation">…</div>
    ▶<ins class="adsbygoogle" data-adsbygoogle-status="done" style="display: none;">…</ins>
    ▶<script type="text/javascript">…</script>
    ▶<div class="container-fluid middle-body">…</div>
    ▶<div class="container sponsor">…</div>
    ▶<div class="container concept">…</div>
    ▶<div class="footer">…</div>
     <!-- JS -->
     <script src="http://share.olanlab.com/asset/js/jquery-1.11.2.min.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/plugin/bootstrap/js/bootstrap.min.js"></script>
     <!-- TEMPLATE -->
     <script src="http://share.olanlab.com/asset/js/jquery.highlight.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/js/handlebars.js" type="text/javascript"></script>
     <!-- FORM -->
     <script src="http://share.olanlab.com/asset/plugin/jquery-easy-ui/jquery.easyui.min.js" type="text/
     javascript"></script>
     <script src="http://share.olanlab.com/asset/plugin/jquery-easy-ui/locale/easyui-lang-th.js" type=
     "text/javascript"></script>
     <script src="http://share.olanlab.com/asset/js/jquery.easing.1.3.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/plugin/ckeditor/ckeditor.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/plugin/ckfinder/ckfinder.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/js/jquery.blockUI.js" type="text/javascript"></script>
     <script src="http://share.olanlab.com/asset/plugin/bootstrap/js/bootstrap-tagsinput.min.js" type=
     "text/javascript"></script>
     <script src="http://share.olanlab.com/asset/plugin/bootstrap/js/typeahead.min.js" type="text/
     javascript"></script>
     <script src="http://share.olanlab.com/asset/js/jquery.form.utility.js" type="text/javascript">
     </script>
     <script src="http://share.olanlab.com/asset/frontend/plugin/select2/js/select2.min.js" type="text/
     javascript"></script>
    ▶<div id="fb-root" class=" fb_reset">…</div>
```

html | body

Styles | Computed | Event Listeners »

Filter :hov .cls +

```
element.style {
}
body {                                   it.min.css:692
}
body {                                   it.min.css:143
  font-family: "Helvetica
    Neue",Helvetica,Arial,sans-serif;
  font-size: 14px;
  line-height: 1.42857143;
  color: ■#333;
  background-color: □#fff;
}
body {                                   it.min.css:130
  margin:▶0;
}
* {                                      it.min.css:143
  -webkit-box-sizing: border-box;
  -moz-box-sizing: border-box;
  box-sizing: border-box;
}
body {                                   user agent stylesheet
  display: block;
  margin:▶8px;
}
Inherited from html
html {                                   it.min.css:143
  font-size: 10px;
  -webkit-tap-highlight-color: □rgba(0,0,0,0);
}
html {                                   it.min.css:130
  font-family: sans-serif;
  -webkit-text-size-adjust: 100%;
  -ms-text-size-adjust: 100%;
```

Console | What's New ×

Highlights from the Chrome 63 update

Multi-client remote debugging
Use DevTools while debugging your app from an IDE or testing framework.

Workspaces 2.0
Save changes made in DevTools to disk, now with more helpful UI and better auto-mapping.

Four new audits

share.olanlab.com/th

Share

เรียนรู้ แบ่งปันกับ Share

เคล็ดลับและเทคนิค

แนะนำ 9 เว็บไซต์แจกรูปฟรีแบบไม่ติดลิขสิทธิ์

แนะนำเว็บไซต์แจกรูปฟรี แบบไม่ติดลิขสิทธิ์สำหรับนักพัฒนาเว็บ (Web Developer) นักทำเนื้อหา (Content Creator) กราฟฟิค (Graphic)

Sep 10, 2017

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Audits

top | Filter | Default levels

Console

Console | What's New

Highlights from the Chrome 63 update

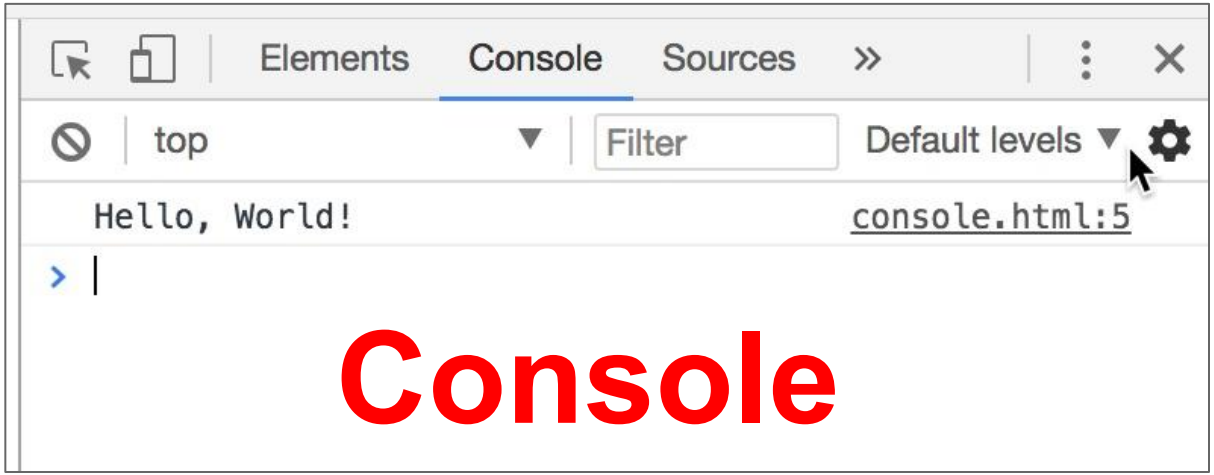
Multi-client remote debugging
Use DevTools while debugging your app from an IDE or testing framework.

Workspaces 2.0
Save changes made in DevTools to disk, now with more helpful UI and better auto-mapping.

Four new audits

console.log

```
<script>
    console.log('Hello, world!');
</script>
```



Variable

เก็บข้อมูลด้วยตัวแปร

Variable

Most of the time, a JavaScript application needs to work with information.

A variable is a “named storage” for data. We can use variables to store goodies, visitors and other data.

```
let user = 'John';
let age = 25;
let message = 'Hello';
let admin = true;
let address;
```

Variable Naming

1. The name must contain only letters, digits, symbols `$` and `_`.
2. The first character must not be a digit.

Correct Variable names

```
let userName;
let test123;
```

Incorrect Variable names

```
let 1st; // cannot start with a digit
let my-name; // a hyphen '-' is not allowed ain the name


let let = 5; // can't name a variable "let", error!
let return = 5; // also can't name it "return", error!
```

Data Types

1. Boolean
2. Null
3. Undefined
4. String
5. Number

```
let admin = true;
let message = null;
let address;
let user = 'John';
let age = 25;
```

Constant

To declare a constant (unchanging) variable, one can use `const` instead of `let`:

```
const myBirthday = '18.04.1982';

const COLOR_RED = "#F00";
const COLOR_GREEN = "#0F0";
const COLOR_BLUE = "#00F";
const COLOR_ORANGE = "#FF7F00";
```

Error

```
const myBirthday = '18.04.1982';
myBirthday = '01.01.2001'; // error, can't reassign the constant!
```

Operators

กำหนด จัดการ ข้อมูลด้วย Operators

Assignment Operators

assignment operator which assigns a value to a variable.

```
let x = 1;
let y = -x;
alert(y); // -1
```

```
let x = y = 1;
alert(x); // 1
alert(y); // 1
```

```
let x = 1, y = 3;
alert(x); // 1
alert(y); // 3
```

Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)
++	Increment
--	Decrement

Arithmetic Operators

```
let x = 4;
let y = 2;
console.log(x + y) // ADDITION        : 6
console.log(x - y) // SUBTRACTION         : 2
console.log(x * y) // MULTIPLICATION  : 8
console.log(x / y) // DIVISION        : 2
console.log(x % y) // MODULUS         : 0
console.log(x ** y) // POWERS         : 16
```

Arithmetic + Assignment Operators

```
let a, b, c;
a = b = c = 2 + 2;
alert(a); // 4
alert(b); // 4
alert(c); // 4
```

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Arithmetic + Assignment Operators

```
let n = 2;
n = n + 5;
n = n * 2;
alert(n);
```

Shorthand

```
let m = 2;
m += 5; // now m = 7 (same as m = m + 5)
m *= 2; // now m = 14 (same as m = m * 2)
alert(m); // 14
```

```
let o = 2;
o *= (3 + 5);
alert(o); // 16  (right part evaluated first, same as o *= 8)
```

Operator precedence

determines the way in which operators are parsed with respect to each other. Operators with higher precedence become the operands of operators with lower precedence.

2 1

```
let x = 1 + 2 * 2;
alert(x); // 5
```

3 2 1

```
let x = 1;
let y = 2;
let z = 3 - (x = y + 1);
alert(x); // 3
alert(y); // 2
alert(z); // 0
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Operator_Precedence

Increment : Postfix

```
let counter1 = 2;
counter1++;          // works same as counter = counter + 1, but shorter
alert(counter1);     // 3
```

```
let counter4 = 1;
let a2 = counter4++;
alert(a2); // 1
```

Increment : Prefix

```
let counter3 = 1;
let a1 = ++counter3;
alert(a1); // 2
```

Decrement : Postfix

```
let counter2 = 2;
counter2--;       // works same as counter = counter - 1, but shorter
alert(counter2); // 1
```

```
let counter5 = 1;
let a3 = counter5--;
alert(a3); // 1
```

Decrement : Prefix

```
let counter6 = 1;
let a4 = --counter6;
alert(a4); // 0
```

String Operators

String concatenation

```
let s = "my" + "string";
alert(s); // mystring
```

```
alert('1' + 2); // "12"
alert(2 + '1'); // "21"
```

Subtraction and Division convert to numbers

```
alert(2 - '1'); // 1
alert('6' / '2'); // 3
```

String Methods

```
var txt = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
var sln = txt.length; //

var str = "HELLO WORLD";
var res = str.charAt(0);

var str = "Please locate where 'locate' occurs!";
var pos = str.indexOf("locate");

var str = "Please locate where 'locate' occurs!";
var pos = str.lastIndexOf("locate");

var str = "Visit Microsoft!";
var res = str.replace("Microsoft", "OlanLab");
```

Comparison Operators

- Greater/less than: `a > b`, `a < b`.
- Greater/less than or equals: `a >= b`, `a <= b`.
- Equality check is written as `a == b`
- Not equals. In maths the notation is ≠, in JavaScript it's written as an assignment with an exclamation sign before it: `a != b`.

*Just as all other operators, **a comparison returns a value**. **The value is of the boolean type.***

- `true` – means "yes", "correct" or "the truth".
- `false` – means "no", "wrong" or "a lie".

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

Number comparisons

```
alert(2 > 1);  // true (correct)
alert(2 == 1); // false (wrong)
alert(2 != 1); // true (correct)
```

```
let result = 5 > 4; // assign the result of the comparison
alert(result); // true
```

String comparisons

```
alert('Z' > 'A'); // true
alert('Glow' > 'Glee'); // true
alert('Bee' > 'Be'); // true
```

Comparisons of different types

```
alert('2' > 1); // true, string '2' becomes a number 2
alert('01' == 1); // true, string '01' becomes a number 1
```

```
alert(true == 1); // true
alert(false == 0); // true
```

Strict equality

```
alert(0 == false); // true
alert(0 == false); // true

alert(0 === false); // false, because the types are different
```

null

```
alert(null > 0);          // false
alert(null == 0);         // false
alert(null >= 0);         // true
```

undefined

```
alert(undefined > 0);     // false
alert(undefined < 0);     // false
alert(undefined == 0);    // false
```

null vs undefined

```
alert(null === undefined);    // false
alert(null == undefined);     // true
```

Logical Operators

There are three logical operators in JavaScript: `||` (OR), `&&` (AND), `!` (NOT).

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
||	or	(x == 5 || y == 5) is false
!	not	!(x == y) is true

|| (OR)

```
alert(true || true);   // true
alert(false || true);  // true
alert(true || false);  // true
alert(false || false); // false
```

&& (AND)

```
alert(true && true);   // true
alert(false && true);  // false
alert(true && false);  // false
alert(false && false); // false
```

! (NOT)

```
alert(!true); // false
alert(!0); // true
```

Control Structures

A Control Structure is a block of programming that **analyzes variables** and chooses a direction in which to go based on given parameters.

Conditional Statement

1. if
2. Ternary
3. Switch

Loop Statement

1. While
2. Do … while
3. For

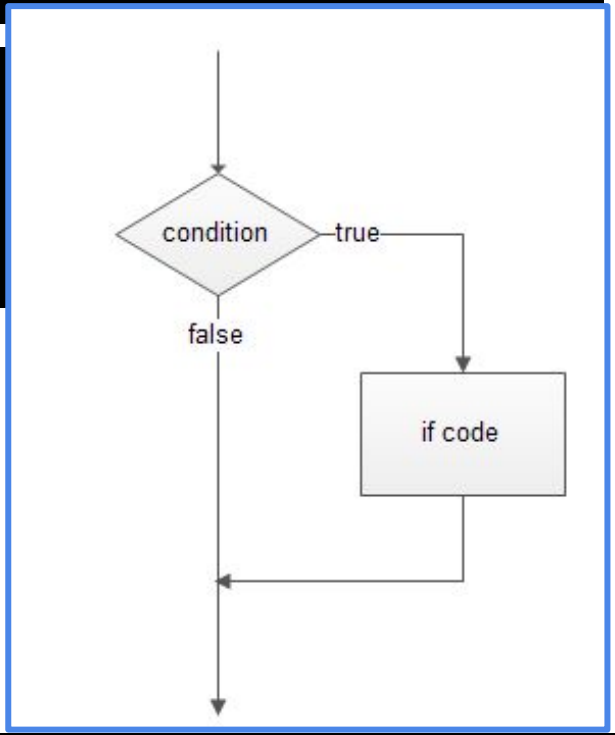
Conditional Statement

การเลือกทำอย่างใดอย่างหนึ่ง

if Statement

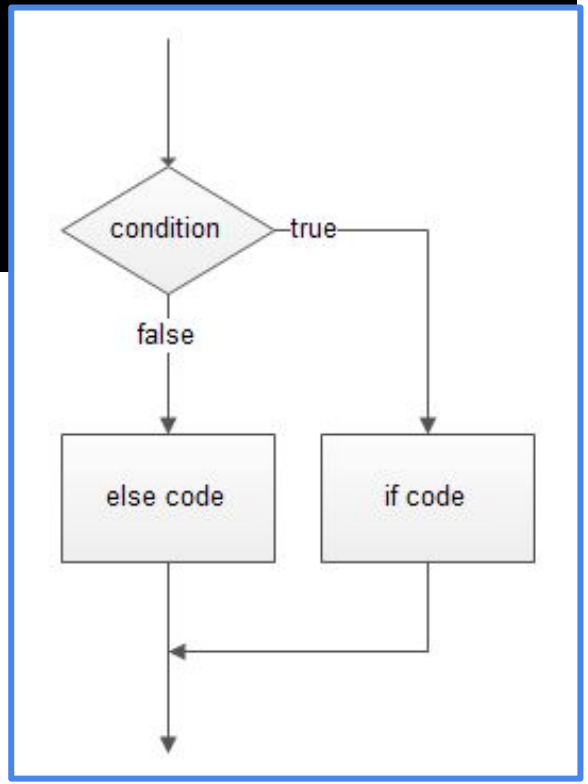
```
let year = prompt('In which year was ECMAScript-2015 specification published?', '');
if (year == 2015) alert('You are right!');
```

```
if (year == 2015) {
    alert("That's correct!");
    alert("You're so smart!");
}
```



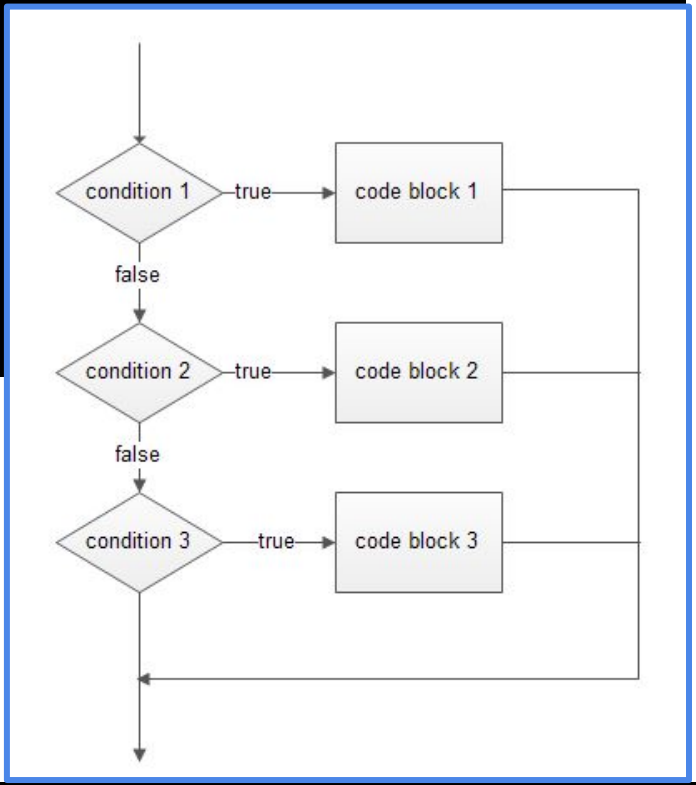
else Statement

```
let year2 = prompt('In which year was ECMAScript-2015 specification published?', '');
if (year2 == 2015) {
    alert('You guessed it right!');
} else {
    alert('How can you be so wrong?');
}
```



else if Statement

```
let year3 = prompt('In which year was ECMAScript-2015 specification published?', '');
if (year3 < 2015) {
    alert('Too early...');
} else if (year3 > 2015) {
    alert('Too late');
} else {
    alert('Exactly!');
}
```



Ternary Operator

*is an **operator** that takes three arguments. The first argument is a comparison argument, the second is the result upon a true comparison, and the third is the result upon a false comparison.*

```
let accessAllowed = (age > 18) ? true : false;
```

```
let accessAllowed = age > 18;
```

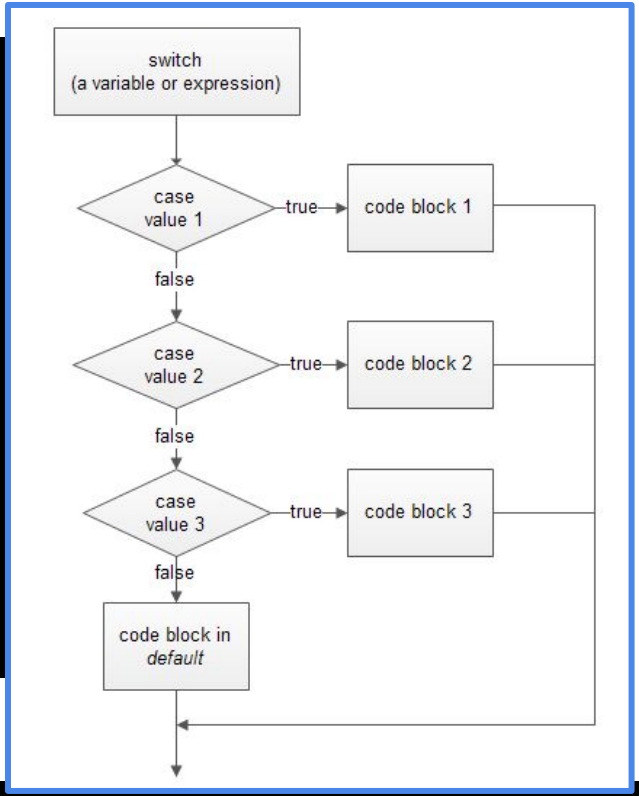
Nested Ternary Operator

```
let age = prompt('age?', 18);
let message =  (age < 3) ? 'Hi, baby!' :
            (age < 18) ? 'Hello!' :
            (age < 100) ? 'Greetings!' : 'What an unusual age!';
alert(message);
```

The “switch” statement

It gives a **more descriptive way** to **compare a value with multiple variants.**

```
let a = 2 + 2;
switch (a) {
    case 3:
        alert('Too small');
          break;
    case 4:
        alert('Exactly!');
          break;
    case 5:
        alert('Too large');
        break;
    default:
        alert("I don't know such values");
}
```



switch when No break

```
let b = 2 + 2;
switch (b) {
    case 3:
        alert('Too small');
    case 4:
        alert('Exactly!');
    case 5:
        alert('Too big');
    default:
        alert("I don't know such values");
}
```

If there is no **break** then the execution continues with the next **case** without any checks.

Group cases

```
let c = 2 + 2;
switch (c) {
    case 4:
        alert('Right!');
        break;
    case 3: // (*) grouped two cases
     case 5:
        alert('Wrong!');
        alert("Why don't you take a math class?" );
        break;
    default:
        alert('The result is strange. Really.' );
}
```

Loop Statement

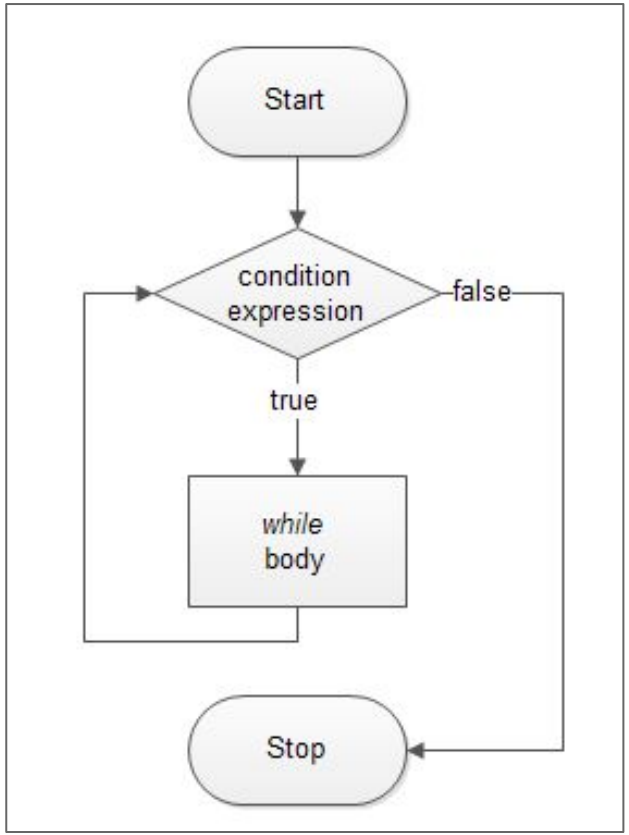
การทำซ้ำจนกว่าเงื่อนไขไม่เป็นจริง

```
console.log(0 + '%');
console.log(10 + '%');
console.log(20 + '%');
console.log(30 + '%');
console.log(40 + '%');
console.log(50 + '%');
console.log(60 + '%');
console.log(70 + '%');
console.log(80 + '%');
console.log(90 + '%');
console.log(100 + '%');
```

```
let i = 0;
while (i <= 100) {
    console.log(i + '%');
    i += 10;
}
```

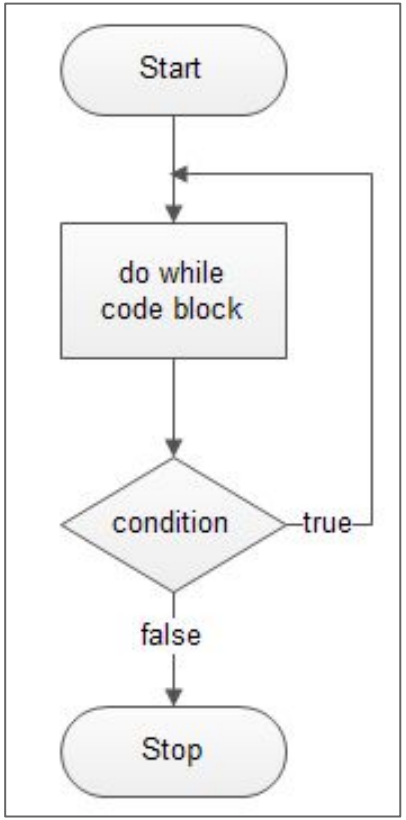
While Statement

```
let i = 0;
while (i < 3) {  // shows 0, then 1, then 2
    alert(i);
    i++;
}
```



Do … While Statement

```
let j = 0;
do {
    alert(j);
    j++;
} while (j < 3);
```



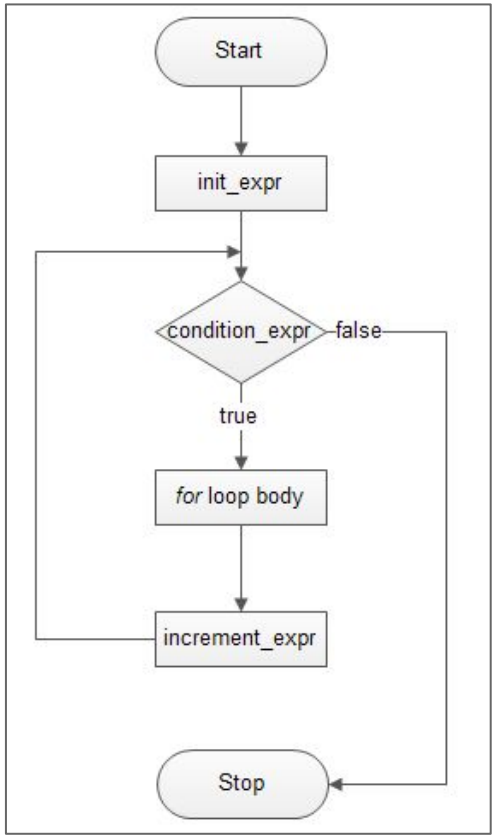
For Statement

Initialize

Increment or Decrement

```
for (let i = 0; i < 3; i++) { // shows 0, then 1, then 2
     alert(i);
}
```

Condition



#1 No Intialize

```
let j = 0; // we have i already declared and assigned
for (; j < 3; j++) { // no need for "begin"
    alert(j); // 0, 1, 2
}
```

#2 Only Condition

```
let k = 0;
for (; k < 3;) {
    alert(k++);
}
```

#3 No at all (Infinite Loop)

```
for (; ;) {
    // repeats without limits
}
```

break

The break statement "jumps out" of a loop.

```
let sum = 0;
while (true) {
   let value = prompt("Enter a number");
   if (!value) break;
   sum += parseInt(value);
}
alert('Sum: ' + sum);
```

continue

The continue statement "jumps over" one iteration in the loop.

```
for (let i = 0; i < 10; i++) {
    // if true, skip the remaining part of the body
    if (i % 2 == 0) continue;
    alert(i); // 1, then 3, 5, 7, 9
}
```

Arrays

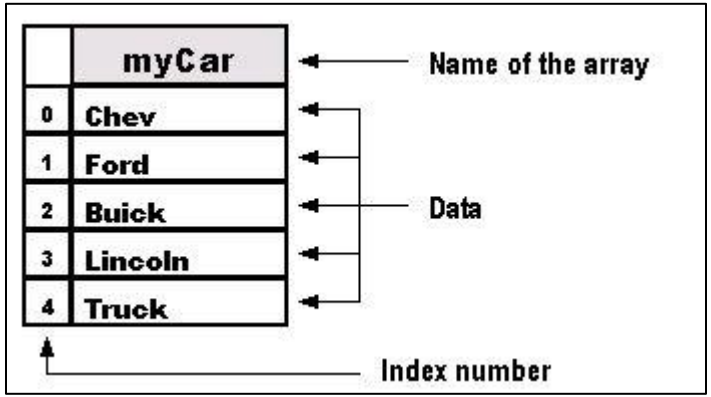
Arrays are used to store multiple values in a single variable.

Creating an Empty Array

```
let arr = new Array();
let arr2 = [];
```

Initialize an Array

```
let myCar = ["Chev", "Ford",
             "Buick", "Lincoln",
"Truck"];
```



Replace or Add Element in Array

```
myCar[2] = 'Toyota'; // now ["Chev", "Ford", "Toyota", "Lincoln", "Truck"]
myCar[5] = 'BMW'; // now ["Chev", "Ford", "Buick", "Lincoln", "Truck", "BMW"]
```

Store elements of Any Type

```
let arr = [ 'Apple', { name: 'John' }, true, function() { alert('hello'); } ];

alert(arr[1].name); // John

arr[3]() ; // hello
```

Loops with Arrays

```
let fruits = ["Apple", "Orange", "Pear"];
for (let i = 0; i < fruits.length; i++) {
    alert(fruits[i]);
}
```

Iterates over **array elements**

```
let fruits2 = ["Apple", "Orange", "Plum"];
for (let fruit of fruits2) {
    alert(fruit);
}
```

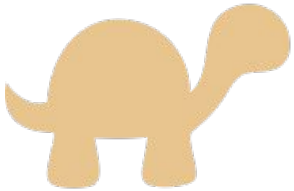
Iterates over **key of elements**

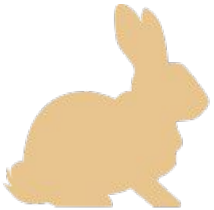
```
let arr4 = ["Apple", "Orange", "Pear"];
for (let key in arr4) {
    alert(arr4[key]); // Apple, Orange, Pear
}
```

Iterates over **key of elements**

```
let arr5 = [];
arr5["NY"] = "New York";
arr5["LD"] = "London";
for (let key in arr5) {
   console.log(key + ':' + arr5[key]); // NY:New York, LD:London
}
```

Methods pop/push, shift/unshift

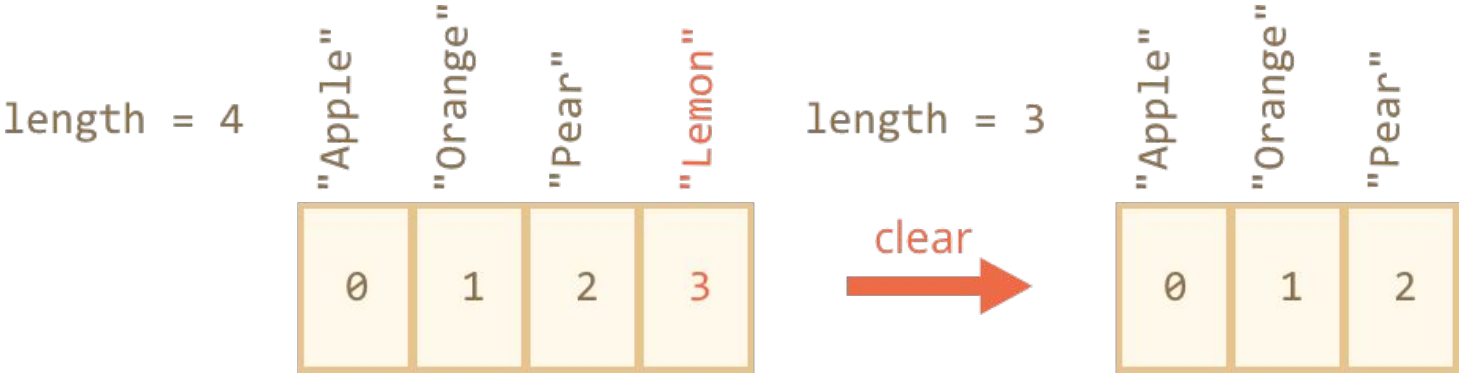




SLOW

FAST

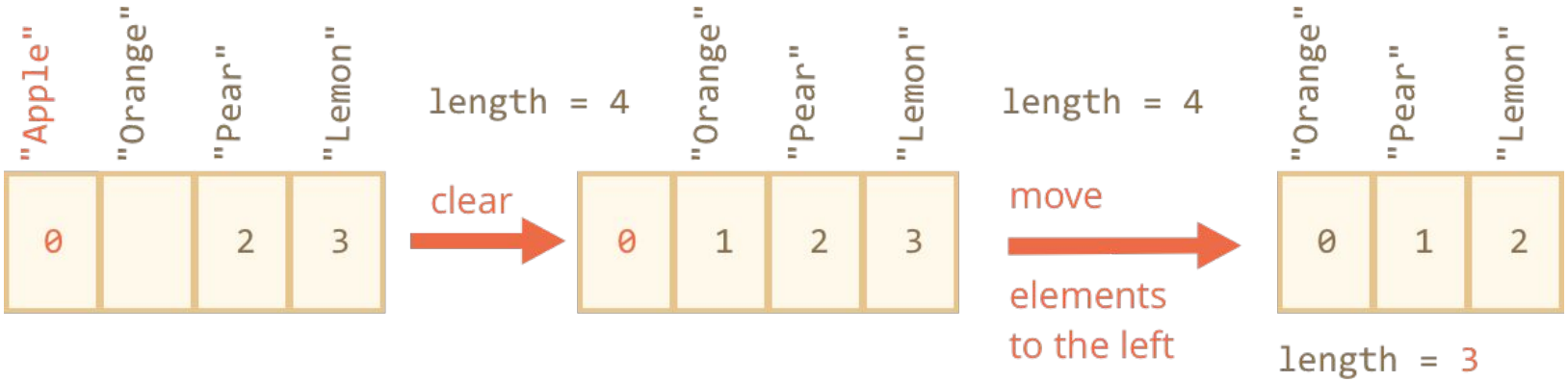
pop / push




```
let fruits1 = ["Apple", "Orange", "Pear", "Lemon"];
alert(fruits1.pop()); // remove "Lemon" and alert it
alert(fruits1); // Apple, Orange, Pear
```

```
let fruits2 = ["Apple", "Orange"];
fruits2.push("Pear");
alert(fruits2); // Apple, Orange, Pear
```

shift / unshift




```
let fruits3 = ["Apple", "Orange", "Pear", "Lemon"];
alert(fruits3.shift()); // remove Apple and alert it
alert(fruits3); // Orange, Pear, Lemon
```

```
let fruits4 = ["Orange", "Pear"];
fruits4.unshift('Apple');
alert(fruits4); // Apple, Orange, Pear
```

```
let a1 = 20;
let a2 = a1;
a1 = 10; // a1 = 10, a2 = 20
```

Cloning Array

```
let arr1 = ["Apple"];
let arr2 = [...arr1]; // Create new Array with arr
let arr3 = arr1; // Reference to arr

arr1.push("Banana");
alert(arr1); // ["Apple", "Banana"]
alert(arr2); // ["Apple"]
alert(arr3) // ["Apple", "Banana"]
```

Array is Reference Types

Objects

Objects are used to store collections of data and more complex entities.

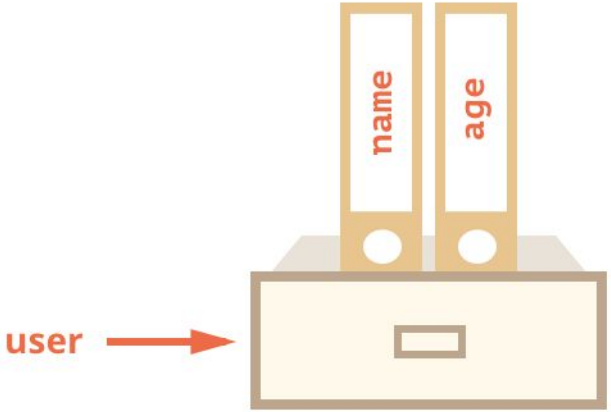
```
let admin = {
    firstname: "Olan",
    lastname: "Samritjiarapon",
    age: 33,
    gender: "male"
}
```

```
let guest = {
    firstname: "Seen",
    lastname: "Vechprasit",
    age: 33,
    gender: "female"
}
```

Key **Value**

Object Initializer

```
let user = {      // an object
    name: "John",  // by key "name" store value "John"
    age: 30        // by key "age" store value 30
};
```



Creating an Empty Object

```
let user1 = new Object(); // "object constructor" syntax
let user2 = {};  // "object literal" syntax
```

Set Object Properties

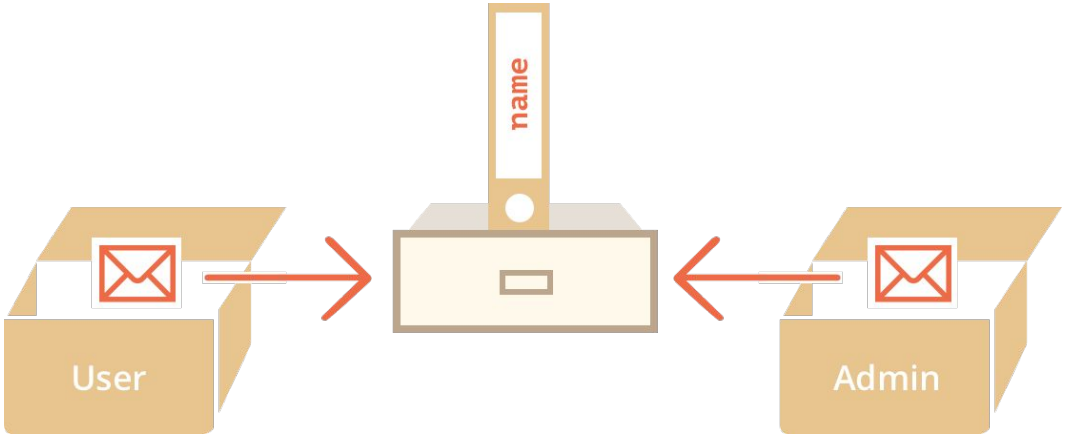
```
user1.name = "Jack";
user2.name = "Rose";
```

Remove Object Properties

```
delete user1.name;
```

Copying by reference

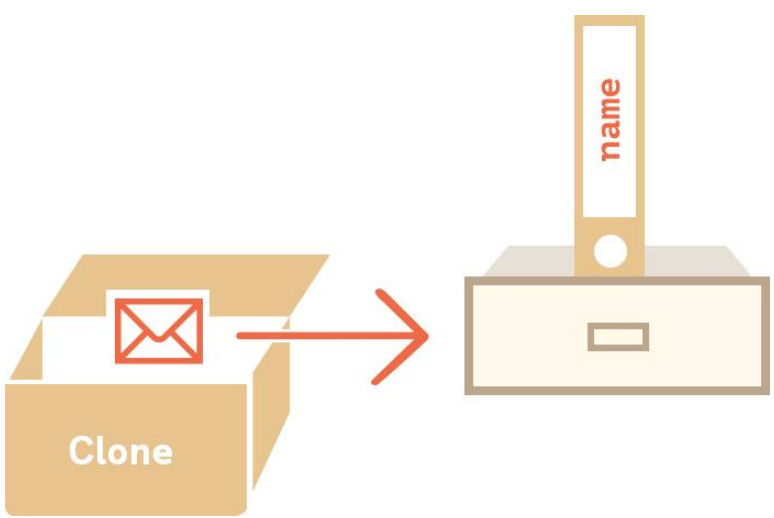
```
let user = { name: "John" };
let admin = user; // copy the reference
```

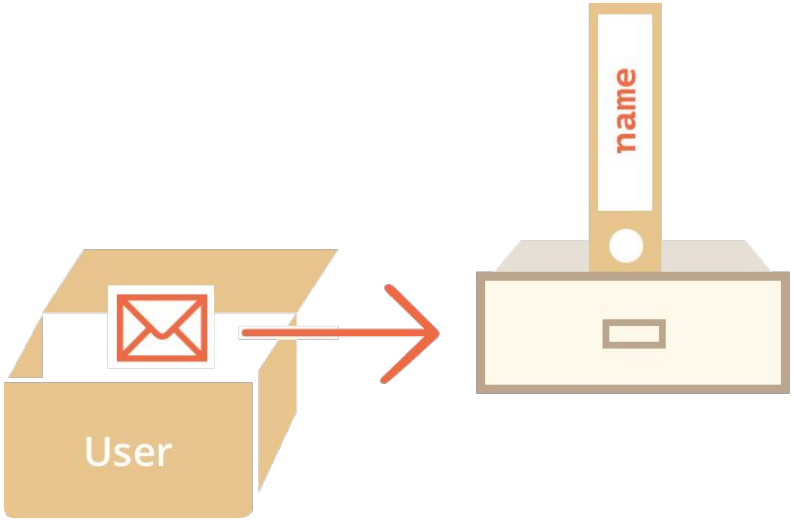


Object is Reference Types

Cloning Object

```
let clone = Object.assign({}, user);
```





Functions

เปลี่ยนโค้ดที่เขียนซ้ำๆ มาเป็น Functions

Functions

ฟังก์ชั่น คือ ชุดคำสั่งที่สามารถประมวลผล และ คืนผลลัพธ์จากการประมวลผลนั้นสู่โปรแกรมหลักได้ ซึ่งจำเป็นในการเขียนโปรแกรมเพื่อช่วยลดการเขียนโค้ดซ้ำซ้อน จัดระเบียบ และช่วยเพิ่มประสิทธิภาพโดยฟังก์ชั่น ของ Javascript มี 2 ประเภท คือ

1. **ฟังก์ชั่นที่ผู้ใช้สร้างขึ้นมาเอง**

   ผู้ใช้สามารถสร้างฟังก์ชั่นขึ้นมาใหม่ ทำได้โดยง่าย โดยอาศัยโครงสร้างพื้นฐานตัวแปร ค่าคงที่ โอเปอเรเตอร์ และการควบคุมโปรแกรมที่กล่าวมาแล้ว มาใช้ในการสร้างฟังก์ชั่น

2. **ฟังก์ชั่นที่มากับ Javascript**

   ฟังก์ชั่นที่ Javascript มีมาให้ใช้งานเพื่อความสะดวกของนักพัฒนา

Creating a Function

function's name

```
function showMessage() {
    alert('Hello everyone!');
}
```

Invoking a Function

```
showMessage();
```

Local Variables

*Variables declared within a JavaScript function, become **LOCAL** to the function.*
*Local variables have **local scope**: They can only be accessed within the function.*

```
function showMessage1() {
    let message1 = "Hello, I'm JavaScript!"; // local variable
    alert(message1);
}
showMessage1(); // Hello, I'm JavaScript!
alert(message1); // <-- Error! The variable is local to the function
```

Global Variables

*A variable declared outside a function, becomes **GLOBAL**.*
*A global variable has **global scope**: All scripts and functions on a web page can access it.*

```
let userName2 = 'John';
function showMessage2() {
    let message2 = 'Hello, ' + userName2;
    alert(message2);
}
showMessage2(); // Hello, John
```

Local VS Global Variables

```
let userName3 = 'John';
function showMessage3() {
    let userName3 = "Bob";                // declare a local variable
    let message3 = 'Hello, ' + userName3; // Bob
    alert(message3);
}
showMessage3(); // the function will create and use it's own userName
alert(userName3);
// John, unchanged, the function did not access the outer variable
```

Passing Arguments by Value

Local Variables

```
function showMessage(from, text) { // arguments: from, text
    alert(from + ': ' + text);
}
```

```
showMessage('Ann', 'Hello!'); // Ann: Hello!
showMessage('Ann', "What's up?"); // Ann: What's up?
```

Default Argument Values

Default Values

```
function showMessage2(from, text = "no text given") {
    alert(from + ": " + text);
}
```

```
showMessage2("Ann"); // Ann: no text given
```

The return Statements

```
function sum(a, b) {
    return a + b;
}
let result = sum(1, 2);
alert(result); // 3
```

Early exit from function

```
function showMovie(age) {
    if (!checkAge(age)) {
        return;
    }
    alert("Showing you the movie"); // (*)
}
```

A function without it returns

```
function doNothing2() { return; }
alert(doNothing2() === undefined); // true
```

A function with a empty return

```
function doNothing1() { /* empty */ }
alert(doNothing1() === undefined); // true
```